K L (DEEMED TO BE) UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

PROJECT REPORT

ON

PARALLEL FILE SEARCH USING THREADS

A Multithreaded File Search Application Using C++17 and Qt 6

A project report submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by

D Pawan Kumar — **2520090102**

Ajith Reddy — **2520090186**

Hitesh Suthar — **2520090078**

Under the guidance of

Raghupathi M

CSIT Department

K L (Deemed to be) University

Hyderabad, Telangana | Academic Year: 2026–2027

DECLARATION

We hereby declare that the project report entitled “Parallel File Search Using Threads” is a record of work carried out as part of the academic project in partial fulfillment of the requirements for the award of the Bachelor of Technology degree in Computer Science and Engineering. The report describes the design and implementation of a C++17 desktop application that performs concurrent directory traversal and file-content searching using a thread pool and a Qt 6 graphical interface.

The report has been prepared from the project source code and documentation supplied for review. Student names, roll numbers, faculty guide, and academic year must be completed and verified before submission.

D Pawan Kumar — **2520090102**

Ajith Reddy — **2520090186**

Hitesh Suthar — **2520090078**

CERTIFICATE

This is to certify that the project report entitled “Parallel File Search Using Threads” is a bonafide record of project work prepared by the students named below, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering.

The project implements a desktop file-search application using C++17, the C++ Standard Library concurrency facilities, CMake, and Qt 6 Widgets. This report is submitted for academic evaluation, subject to verification of implementation and institutional details.

ACKNOWLEDGEMENT

We express our sincere gratitude to our faculty guide and the Department of Computer Science and Engineering for their guidance and support during the development of this project. Their encouragement helped us connect operating-system concepts such as concurrency, synchronization, thread lifecycle management, and file-system operations with a practical software application.

We also thank the course coordinator, laboratory staff, classmates, and everyone who provided suggestions or assistance. Finally, we are grateful to our families for their continued encouragement throughout the project.

ABSTRACT

Parallel File Search Using Threads is a desktop application designed to locate matching filenames and text within files across a directory tree. Searching a large collection of files sequentially can be time-consuming, while performing file operations on a graphical user-interface thread can make an application appear unresponsive. This project separates the graphical interface from a concurrent search engine.

The application is implemented in C++17 with Qt 6 Widgets. Its core library uses std::thread, std::mutex, std::condition_variable, std::atomic, std::future, and std::filesystem. A coordinator discovers files and schedules batches of work to a reusable thread pool. Worker tasks inspect filenames and, when requested, read text files incrementally line by line. The search supports case-sensitive and case-insensitive matching, regular expressions, recursive traversal, binary-file detection, and a maximum file-size guard of 50 MB.

The interface provides search configuration, progress statistics, active-worker information, a sortable results table, an in-memory quick filter, context-menu actions, activity logs, and CSV export. CSV output uses RFC 4180-style escaping and UTF-8 BOM encoding. The project includes CTest-managed test targets for thread-pool behavior, search operations, cancellation, and export handling. Test outcomes should be confirmed by running the tests in the target environment.

Keywords: C++17, multithreading, thread pool, synchronization, Qt 6, filesystem traversal, concurrent search, operating systems.

TABLE OF CONTENTS

Acknowledgement

Abstract

1. Introduction
   1.1 Background of the Project
   1.2 Problem Statement
   1.3 Scope and Objectives
   1.4 Importance and Target Users
2. OS Concepts and Background Study
3. System Requirements
4. Technology and Tools Used
5. System Architecture and Design
6. Implementation
7. Execution and Demonstration
8. Conclusion
9. References

CHAPTER 1 — INTRODUCTION

1.1 Background of the Project

File search is a common task for developers, system administrators, researchers, and everyday computer users. Users may need to locate a file by its name or identify lines of text within source code, logs, notes, and documents. As directory trees grow, sequential scans can become slow because they must visit many directory entries and open many files.

Parallel File Search uses a coordinator and a pool of worker threads. The coordinator discovers candidate files and submits batches to workers, which process independent files while the graphical interface remains responsive.

1.2 Problem Statement

The project addresses slow sequential file inspection, interface freezes caused by blocking work on the UI thread, and the synchronization, cancellation, progress-reporting, and resource-management challenges introduced by concurrent execution.

1.3 Scope and Objectives

- Search filenames, file contents, or both.
- Traverse directories recursively or inspect only immediate entries.
- Support case-sensitive and case-insensitive matching and regular expressions.
- Use a configurable worker pool and batch discovered paths.
- Detect likely binary files and skip files larger than 50 MB.
- Display results and support filtering, context actions, logging, and CSV export.
- Demonstrate thread coordination, synchronization, cancellation, and clean shutdown.

1.4 Importance and Target Users

The application is useful for developers searching source repositories, administrators reviewing log files, students learning operating-system concepts, and users locating text across folders. Its academic value is in demonstrating how concurrency can improve responsiveness while requiring careful shared-state and thread-lifecycle design.

CHAPTER 2 — OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating-System Concepts

2.1.1 Threads and Concurrency

Threads are execution paths within a process. They share process resources but have individual execution state. The application uses a coordinator and worker pool so file processing does not block the GUI thread.

2.1.2 Mutual Exclusion

Mutexes protect shared collections and task-queue operations. Critical sections should be short; file reading and matching should not hold a global results lock.

2.1.3 Condition Variables and Task Queues

Workers wait on a condition variable when the task queue is empty and are notified when tasks are added or shutdown begins, avoiding busy-waiting.

2.1.4 Atomic Variables

Atomic flags and counters represent running state, cancellation requests, discovered/scanned files, match counts, and active-worker information.

2.1.5 File-System Operations

std::filesystem provides directory traversal and path handling, while std::ifstream supports incremental file reading. Permissions, changing paths, symbolic links, and I/O errors can affect traversal.

2.2 Existing and Proposed Approaches

A sequential scan is simple but may underuse available resources. Creating one thread per file can create excessive overhead. This project uses a bounded thread pool, reuses worker threads, and submits batches of 25 paths per task to reduce scheduling overhead.

2.3 Advantages

- Bounds the number of concurrently running worker threads.
- Reduces repeated thread creation and queue overhead.
- Processes file contents incrementally.
- Separates core search logic from GUI code for testing.
- Supports progress reporting, cancellation, and diagnostic logging.

CHAPTER 3 — SYSTEM REQUIREMENTS

3.1 Hardware Requirements

Component	Requirement / Notes
Processor	Modern multi-core CPU; worker count is configurable
Memory	4 GB RAM or more recommended for development
Storage	Space for source, build output, and search targets
Display	Desktop display supporting Qt Widgets

3.2 Software Requirements

- C++17-compatible compiler such as GCC, Clang, or MSVC.
- Qt 6 Core, Gui, and Widgets modules.
- CMake 3.16 or newer, according to project configuration.
- Ninja or another supported build tool.
- CTest for registered automated tests.
- A supported desktop operating system with filesystem and threading support.

3.3 Operating-System Environment

The project uses standard C++ filesystem and threading facilities and Qt 6. The archive includes Windows build artifacts; compiler and Qt installation paths are machine-specific and should be configured on the target system.

3.4 Development Tools

CMake defines a core library named pfs_core, the ParallelFileSearch desktop executable, and test targets. CMAKE_AUTOMOC supports Qt's signal/slot mechanism.

CHAPTER 4 — TECHNOLOGY AND TOOLS USED

4.1 C++17

C++17 provides filesystem paths, threads, mutexes, condition variables, atomics, futures, callable task wrappers, and regular expressions.

4.2 Standard Library Concurrency

- std::thread for coordinator and worker threads.
- std::mutex and lock types for protected shared state.
- std::condition_variable for sleeping workers and task notifications.
- std::atomic for shared flags and counters.
- std::future and std::packaged_task for task results.
- std::filesystem and std::ifstream for traversal and line-based reading.

4.3 Qt 6 Widgets

Qt provides the desktop interface, search controls, results table, progress indicators, and activity log. SearchController mediates between GUI controls and the engine; signals and slots transfer updates to the UI thread.

4.4 CMake, Ninja, and CTest

CMake organizes build targets and dependencies. Ninja can act as the build backend. CTest provides a common interface for the thread-pool, search, cancellation, and CSV-export tests. Test pass/fail status should be confirmed by executing the suite.

CHAPTER 5 — SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall Architecture

Layer / Module	Responsibility
MainWindow	GUI controls, progress, results, filters, context actions
SearchController	Connects UI requests and engine callbacks
SearchEngine	Directory discovery, task scheduling, lifecycle, statistics
ThreadPool	Task queue, worker threads, condition-variable coordination
SearchWorker	Filename/content matching, regex, binary detection
Logger	Timestamped INFO, SUCCESS, WARN, and ERROR messages
ExportManager	CSV output with escaping and UTF-8 BOM
SearchResult / SearchOptions	Data structures for matches and user options

5.2 System Workflow

1. The user selects a directory, pattern, search mode, recursion, case sensitivity, regex mode, and worker count.
2. The controller passes options to the engine.
3. The coordinator discovers files and groups paths into batches.
4. The thread pool assigns tasks to available workers.
5. Workers match filenames and/or read text line by line.
6. Shared results and counters are updated safely; progress is relayed to the GUI.
7. The user reviews, filters, opens, copies, or exports results.
8. On completion or cancellation, threads are joined cleanly.

5.3 Input and Output Design

Inputs include target directory, pattern, search mode, recursion setting, case sensitivity, regex option, and worker count. Outputs include matching path, match type, line number when relevant, matching text, status, aggregate counters, elapsed time, activity messages, and optional CSV.

CHAPTER 6 — IMPLEMENTATION

6.1 Thread Pool

The ThreadPool maintains worker threads and a queue of callable tasks. A mutex protects queue operations and a condition variable wakes workers when work arrives or shutdown begins. Packaged tasks and futures support tasks whose results need to be collected. Pool ownership is designed to prevent ambiguous copying of thread resources.

6.2 Search Engine and Worker

SearchEngine provides synchronous and asynchronous entry points. The coordinator traverses directories using std::filesystem and schedules batches of file paths. SearchWorker performs filename and content matching according to the selected options. Content is read line by line rather than loading an entire file into memory.

6.3 Matching and File Safety

The search supports ordinary text matching with case options and regular-expression matching through std::regex. Binary detection uses null-byte and non-printable-byte checks. Files larger than 50 MB are skipped to constrain processing costs.

6.4 Synchronization and Cancellation

Atomic flags represent running and cancellation state. Mutexes protect shared results and the task queue; condition variables coordinate task availability. Cancellation should stop discovery and avoid unnecessary pending work, while shutdown joins owned threads.

6.5 GUI, Logging, and Export

The controller relays progress and results to the GUI through Qt signals and slots. The interface displays file counts, matches, active workers, elapsed time, and progress. The logger records timestamped messages. ExportManager writes CSV with escaped fields and a UTF-8 BOM.

6.6 Memory Management

Incremental reading and the file-size limit reduce per-file memory usage. RAII-based locking and standard resource-owning types help manage resources. The result collection can still grow with the number of matches; pagination or streaming export could improve very large searches.

CHAPTER 7 — EXECUTION AND DEMONSTRATION

7.1 Build Procedure

Install Qt 6 and a compatible C++17 toolchain, then run these commands from the project root:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --config Release
ctest --test-dir build --output-on-failure
```

The exact launch path depends on the generator and configuration. A clean build on the submission machine is recommended.

7.2 User Workflow

9. Launch ParallelFileSearch.
10. Choose the target directory.
11. Enter a text or regular-expression pattern.
12. Choose filename-only, content-only, or combined mode.
13. Set recursion, case sensitivity, regex behavior, and worker count.
14. Start searching and observe progress and activity logs.
15. Filter or sort the results table.
16. Open or copy a result, or export results to CSV.

7.3 Application Screenshots

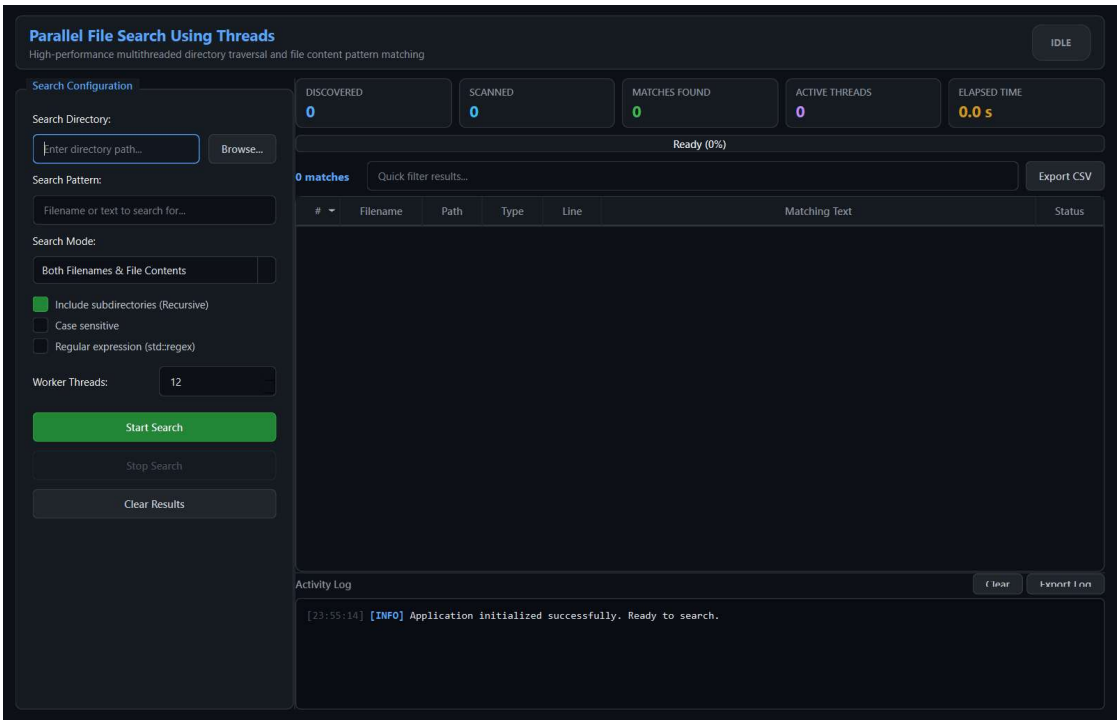


Figure 7.1: Application interface and search configuration.

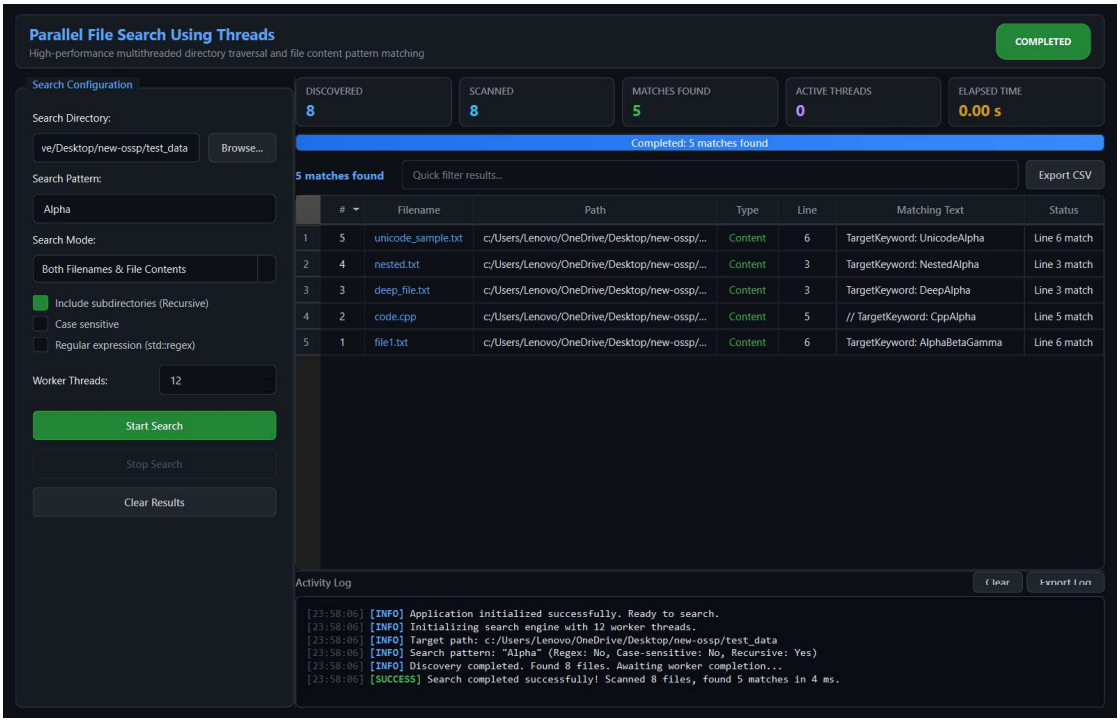


Figure 7.2: Results dashboard after a search.

7.4 Test Plan

Test target	Coverage focus
Thread pool	Task enqueueing, execution, futures, worker lifecycle
Search engine	Pattern matching, search modes, directory traversal
Cancellation	Cancellation request and clean completion
CSV export	Field escaping and UTF-8 BOM output

The project contains corresponding test source files and CTest configuration. This report does not claim that all tests passed because test execution results have not been independently verified.

CHAPTER 8 — CONCLUSION

8.1 Summary and Learning Outcomes

The project combines a Qt desktop interface with a reusable C++17 search core. A coordinator and thread pool distribute file-processing work across worker threads, while synchronized statistics and result storage support progress reporting. Incremental file reading, binary detection, a file-size guard, CSV export, and logging improve usability.

- Applied thread-pool design to a practical filesystem workload.
- Used mutexes, condition variables, atomics, futures, and thread lifecycle management.
- Separated GUI and core logic for modularity and testability.
- Integrated Qt signals and slots for UI updates.
- Practiced CMake-based builds and CTest-based test organization.

8.2 Limitations and Future Work

- Performance depends on disk throughput, file count, caching, and worker count.
- Binary detection is heuristic and may not classify every file correctly.
- Files larger than 50 MB are skipped by design.
- Regular-expression performance depends on pattern and input.
- Large result sets may consume significant memory.
- Future work could add include/exclude filters, pagination, streaming export, benchmarking, and broader error-condition tests.

CHAPTER 9 — REFERENCES

17. C++ reference documentation: std::thread, std::mutex, std::condition_variable, std::atomic, std::future, std::filesystem, and std::regex. https://en.cppreference.com/
18. Qt Documentation: Qt Widgets, Signals & Slots, and Threading Basics. https://doc.qt.io/
19. CMake Documentation: CMake Build System and CTest. https://cmake.org/cmake/help/latest/
20. Project source and documentation supplied for this report: README.md, docs/architecture.md, docs/user_guide.md, CMakeLists.txt, include/, src/, and tests/.
21. RFC 4180: Common Format and MIME Type for Comma-Separated Values (CSV) Files. https://www.rfc-editor.org/rfc/rfc4180